

# Módulo 4: Aprendizaje Automático

## Clase 6: Mini Proyecto y Arquitectura de Pipelines

Diplomado en Ciencia de Datos

10 de julio de 2026

# Objetivos de la Sesión

- Evolucionar de la programación algorítmica a la **Ingeniería de Modelos (MLOps)**.
- Entender el peligro del *Data Leakage* (Fuga de Datos) en el preprocesamiento manual.
- Comprender el concepto y estructura de un **Pipeline** en `scikit-learn`.
- Automatizar la imputación, el escalado y la clasificación en una sola línea de ensamblaje.

# El Problema: Procesamiento Manual

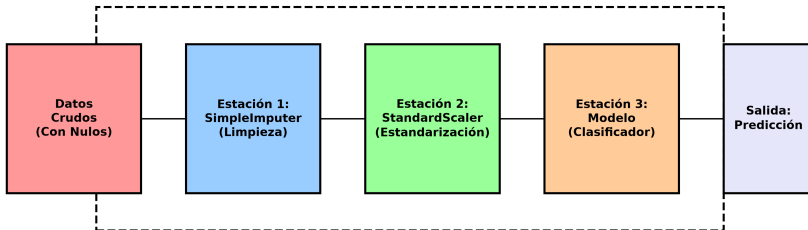
Hasta hoy, hemos construido modelos paso a paso:

- 1 Tomar los datos crudos.
- 2 Rellenar los valores nulos (Imputar).
- 3 Estandarizar los datos (Escalar).
- 4 Entrenar el modelo.

**¿Cuál es el riesgo?** Si aplicamos estas transformaciones de forma manual, es muy fácil olvidar un paso al evaluar nuevos datos, o peor aún, utilizar datos del futuro (Set de Prueba) para calcular promedios de imputación o escalado (**Data Leakage**).

## La Solución: Línea de Ensamblaje (Pipeline)

## Arquitectura de un Pipeline Predictivo en Scikit-Learn



# Anatomía de un Pipeline (Paso a Paso)

Un Pipeline es un objeto que encapsula múltiples pasos secuenciales.

- **Estaciones (Transformers):** Cada paso intermedio debe tener las funciones `fit()` y `transform()` para modificar los datos (ej. Imputador y Escalador).
- **Cierre (Estimator):** El último paso de la cadena debe ser obligatoriamente un modelo predictivo, el cual posee la función `predict()` (ej. Árbol de Decisión).

# Implementación en Python: Importación de Herramientas

Primero, importamos todas las estaciones de nuestra fábrica:

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.tree import DecisionTreeClassifier
```

Cada una de estas clases representa una máquina independiente que debemos ensamblar.

# Implementación en Python: Ensamblaje

Se declara el Pipeline utilizando una lista de tuplas. Cada tupla contiene un nombre identificador y la herramienta instanciada.

```
mi_fabrica = Pipeline(steps=[
    ('limpiador', SimpleImputer(strategy='mean')),
    ('escalador', StandardScaler()),
    ('modelo', DecisionTreeClassifier(max_depth=4))
])
```

**Punto Crítico:** El orden es estrictamente secuencial. Los datos crudos entrarán por 'limpiador' y saldrán por 'modelo'.

# Implementación en Python: Encendido Total

Una vez ensamblado el Pipeline, todo el flujo de trabajo se simplifica drásticamente:

```
# Todo el preprocesamiento y entrenamiento ocurre aquí  
mi_fabrica.fit(X_train, y_train)
```

```
# El preprocesamiento se aplica automáticamente a los  
# datos de evaluación antes de predecir  
predicciones = mi_fabrica.predict(X_test)
```

Esta estructura garantiza que el modelo puede ser puesto en producción y recibir datos crudos de forma segura.



# Fin de la Presentación Teórica

Transición al Mini Proyecto:

`06_Pipelines_Mini_Proyecto.ipynb`

`06_Laboratorio_Evaluacion.ipynb`